# Generative adversarial networks

**A first example: learning to generate numbers**

Directed Reading Program
Florida State University

Data Two networks Two losses Generation

# Learning a function and learning a distribution

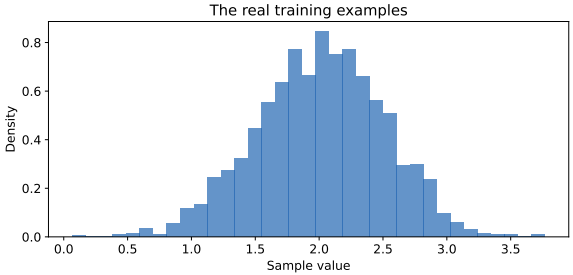
| | Previous lesson | This lesson |
|---|---|---|
| Input | A chosen value $x$ | Random noise $z$ |
| Output | A prediction $f_\theta(x)$ | A generated sample $G_\theta(z)$ |
| Learning signal | A target $\sin(x)$ | A learned discriminator |
| Evaluation | Error at new inputs | Distribution of new samples |

A generated sample has no paired correct answer.

# 1. Real data: a bell-shaped distribution

$$X \sim \mathcal{N}(2, 0.5^2)$$

```
torch.manual_seed(7)
x_train = 2 + 0.5 * torch.randn(
    2048, 1
)
```




2,048 samples, each a single number.

Mean 2, standard deviation 0.5.

We know the distribution so we can check the result.

Direct sampling already works here. The GAN is a teaching example.

# Two networks with different jobs

## Generator $G_\theta$

$$z \sim \mathcal{N}(0, 1), \quad x_{\text{fake}} = G_\theta(z)$$

Transforms noise into candidate samples.

Learns to make the discriminator assign a high real-data probability.

## Discriminator $D_\phi$

$$s = D_\phi(x), \quad d_\phi(x) = \sigma(s)$$

Receives real and generated samples.

Learns to assign probability near 1 to real data and near 0 to generated data.

Each has 49 parameters. We train the two parameter sets separately.

Goodfellow et al. (2014), Generative Adversarial Nets, Section 3.

# 2. The generator architecture

```
G = nn.Sequential(
    nn.Linear(1, 16),
    nn.Tanh(),
    nn.Linear(16, 1)
)
```

$$G_\theta(z) = c + \sum_{j=1}^{16} v_j \tanh(w_j z + b_j)$$

Input: one random number.

Hidden layer: 16 nonlinear responses.

Output: one generated number.

The same architecture as the sine example, with a different training task.

New noise gives new inputs. The same noise and parameters give the same output.

# The discriminator returns a logit

```
D = nn.Sequential(
    nn.Linear(1, 16),
    nn.Tanh(),
    nn.Linear(16, 1)
)
```

$$s = D_\phi(x) \in \mathbb{R}$$

$$d_\phi(x) = \sigma(s) = \frac{1}{1 + e^{-s}}$$

The logit $s$ is unrestricted. Its sigmoid is between 0 and 1.

`BCEWithLogitsLoss` applies sigmoid internally for stable computation.

We pass the raw logits into the loss.

PyTorch: BCEWithLogitsLoss.

# 3. Binary cross-entropy

For a logit $s$ and a label $t \in \{0, 1\}$,

$$\ell(s, t) = -t \log \sigma(s) - (1 - t) \log(1 - \sigma(s)).$$

```
bce = nn.BCEWithLogitsLoss()
```

| Desired label | Predicted real probability | Loss |
|---|---|---|
| $t = 1$ | 0.9 | $-\log(0.9) \approx 0.105$ |
| $t = 1$ | 0.1 | $-\log(0.1) \approx 2.303$ |

Confident predictions of the wrong label incur a larger loss.

# The discriminator learns real versus generated

$$L_D = \frac{1}{m}\sum_i \ell(D_\phi(x_i), 1) + \frac{1}{m}\sum_i \ell(D_\phi(G_\theta(z_i)), 0)$$

- Real training samples have label 1.
- Generated samples have label 0.
- This update changes only the discriminator parameters $\phi$.

The two terms are batch means, added together. This scaling matters when reading the loss plot.

Goodfellow et al. (2014), Generative Adversarial Nets, Section 3.

# The generator learns through the discriminator

$$L_G = \frac{1}{m} \sum_i \ell(D_\phi(G_\theta(z_i)), 1) = -\frac{1}{m} \sum_i \log d_\phi(G_\theta(z_i))$$

- The desired label is 1: the generator wants its samples classified as real.
- This update changes only the generator parameters $\theta$.
- The discriminator supplies a differentiable learning signal.

We use the non-saturating generator objective, rather than the original minimax generator objective.

Goodfellow et al. (2014), Generative Adversarial Nets, Section 3.

# Separate optimizers for separate parameter sets

```
opt_g = torch.optim.Adam(
    G.parameters(), lr=0.001, betas=(0.5, 0.999)
)
opt_d = torch.optim.Adam(
    D.parameters(), lr=0.001, betas=(0.5, 0.999)
)
```

Adam scales updates using running averages of gradients and squared gradients.

The learning rate and `betas` are settings we choose before training.

Each iteration has one discriminator update and one generator update.

# 4. Training: the discriminator step

```
d_history, g_history = [], []
for step in range(6000):
    real = x_train[torch.randint(len(x_train), (128,))]
    fake = G(torch.randn(128, 1))

    D.requires_grad_(True)
    opt_d.zero_grad()
    d_loss = bce(D(real), torch.ones(128, 1))
    d_loss = d_loss + bce(D(fake.detach()), torch.zeros(128, 1))
    d_loss.backward()
    opt_d.step()
```

`detach()` stops this backward pass at the generated samples.

Real batches sample the training set with replacement. These are iterations, not full-data epochs.

# Training: the generator step

Continuation inside the same loop:

```
    D.requires_grad_(False)
    opt_g.zero_grad()
    fake = G(torch.randn(128, 1))
    g_loss = bce(D(fake), torch.ones(128, 1))
    g_loss.backward()
    opt_g.step()

    d_history.append(d_loss.item())
    g_history.append(g_loss.item())
```

Freeze the discriminator parameters, but keep derivatives through its input.

There is no `detach()` here: the gradient must reach the generator.

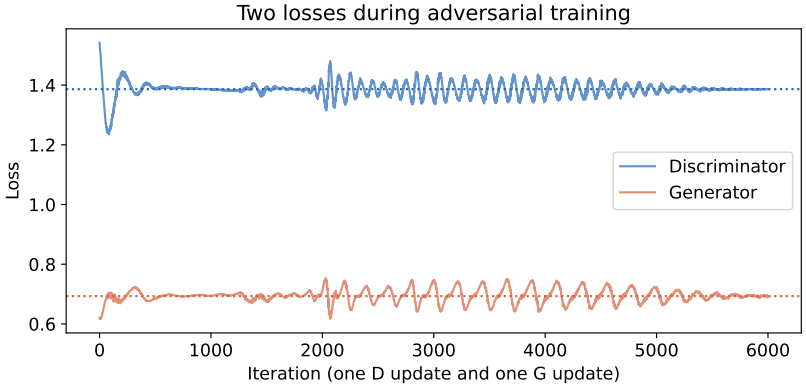
# How the gradient reaches the generator

For one generated sample, let $s = D_\phi(G_\theta(z))$ and $d = \sigma(s)$.

$$\nabla_\theta \ell(s, 1) = \underbrace{(d - 1)}_{\text{loss derivative}} \quad \underbrace{D'_\phi(G_\theta(z))}_{\text{discriminator slope}} \quad \underbrace{\nabla_\theta G_\theta(z)}_{\text{generator derivative}} .$$

- Freezing $\phi$ keeps the discriminator parameters fixed for this update.
- The slope with respect to its input still guides the generator.
- `no_grad()` would break the learning calculation here.

# GAN losses do not need to fall together




At $d = 1/2$, our scaling gives $L_D = 2\log 2 \approx 1.386$ and $L_G = \log 2 \approx 0.693$.

The dotted lines are reference levels. A weak discriminator can reach them with poor samples.

# 5. Generation needs only the generator

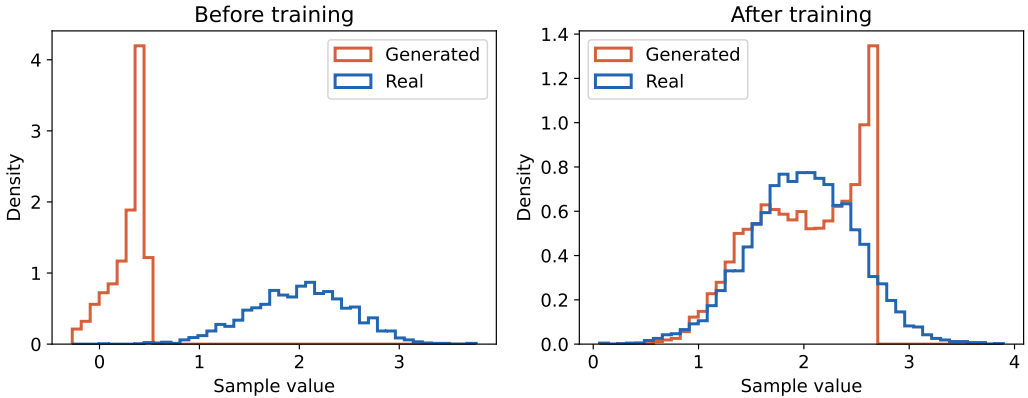
```
torch.manual_seed(2026)
G.eval()
with torch.no_grad():
    real_test = 2 + 0.5 * torch.randn(5000, 1)
    generated = G(torch.randn(5000, 1))
```

These real samples and noise inputs are new, created after training.

Compare the distributions. The first generated sample has no required match with the first real sample.

`no_grad()` is appropriate now: we are evaluating fixed parameters.

# Generated values before and after training




Before: training data and the initial generator. After: fresh real and generated samples.

Each histogram has total area 1. The panels use different vertical scales.

# Numerical checks on 5,000 new samples

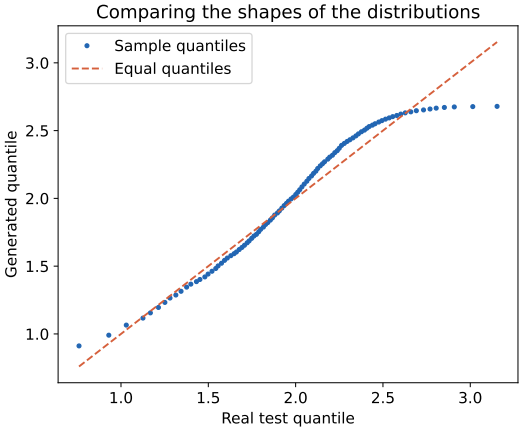
| Statistic | Target | Real test | Generated |
|---|---|---|---|
| Mean | 2.00000 | 1.99510 | 2.00007 |
| Standard deviation | 0.50000 | 0.50366 | 0.49586 |
| 10th percentile | 1.35922 | 1.34386 | 1.31459 |
| Median | 2.00000 | 2.00074 | 2.02866 |
| 90th percentile | 2.64078 | 2.62757 | 2.63094 |

The table compares the theoretical target with two finite sample sets.

Mean and standard deviation describe location and spread. Percentiles examine more of the shape.

Values come from the executed notebook and round to five decimal places.

# Quantiles reveal differences in shape




We compare the 1st through 99th percentiles.

Near the diagonal: similar quantiles.

This run flattens in the upper tail. The histogram also has excess mass near 2.7.

A finite sum of `Tanh` terms is bounded. It cannot exactly reproduce an unbounded normal distribution.

# The next step: generating time series

$$G_\theta(z) = (X_{t_1}, X_{t_2}, \ldots, X_{t_T})$$

- Today's generator outputs one number. A time series generator outputs an entire path or sequence.
- Matching a histogram alone cannot establish temporal dependence.
- Our project will investigate how Volterra signatures can contribute to generation or evaluation.

That method is future work. This example establishes the GAN training pattern.

# Experiments and discussion

1. Change the seed. Does the same schedule give similar samples?
2. Use 1,000 iterations. Compare the losses and the histogram.
3. Use 4 hidden neurons. Does the generated spread change?
4. Try two separated peaks. Does the generator cover both?

Too little variety is mode collapse. Report unsuccessful runs too.

Restart the kernel for each experiment. Use fresh test data after tuning.

References: Goodfellow et al. (2014); PyTorch DCGAN tutorial and BCEWithLogitsLoss.

Companion notebook: `learning_gans.ipynb`